Logistic regression

Please train a logistic regression model to classify iris flowers Features: [sepal_length, sepal_width, petal_length, petal_width]

Labels:
0 : "setosa"
1 : "versicolor"
2 : "virginica"

In [6]:
```
!pip install numpy scikit-learn matplotlib -i https://pypi.tuna.tsinghua.edu.cn/
```

```
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Requirement already satisfied: numpy in e:\anaconda\lib\site-packages (1.26.4)
Requirement already satisfied: scikit-learn in e:\anaconda\lib\site-packages (1.
5.1)
Requirement already satisfied: matplotlib in e:\anaconda\lib\site-packages (3.9.
2)
Requirement already satisfied: scipy>=1.6.0 in e:\anaconda\lib\site-packages (fro
m scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in e:\anaconda\lib\site-packages (fr
om scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in e:\anaconda\lib\site-packa
ges (from scikit-learn) (3.5.0)
Requirement already satisfied: contourpy>=1.0.1 in e:\anaconda\lib\site-packages
(from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in e:\anaconda\lib\site-packages (fro
m matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in e:\anaconda\lib\site-packages
(from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in e:\anaconda\lib\site-packages
(from matplotlib) (1.4.4)
Requirement already satisfied: packaging>=20.0 in e:\anaconda\lib\site-packages
(from matplotlib) (24.1)
Requirement already satisfied: pillow>=8 in e:\anaconda\lib\site-packages (from m
atplotlib) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in e:\anaconda\lib\site-packages
(from matplotlib) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in e:\anaconda\lib\site-packa
ges (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in e:\anaconda\lib\site-packages (from py
thon-dateutil>=2.7->matplotlib) (1.16.0)
```

In [7]:
```
import numpy as np
from sklearn import linear_model
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
```

In [8]:
```
iris = load_iris()
```

In [9]:
```
#complete code here
# describe this dataset (display result from pandas.DataFrame.describe)
import pandas as pd
```

```
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df.describe()
```

Out[9]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

In [10]:

```
#complete code here
#print the dataset and check the data structure
print("Feature data shape:", iris.data.shape)
print("Feature names:", iris.feature_names)
print("Target data shape:", iris.target.shape)
print("Target names:", iris.target_names)
print("Sample data:\n", iris.data[:5])
```

```
Feature data shape: (150, 4)
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'pe
tal width (cm)']
Target data shape: (150,)
Target names: ['setosa' 'versicolor' 'virginica']
Sample data:
 [[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
```

In [11]:

```
#complete code here
#put the data into X and target into y
X = iris.data
y = iris.target
```

In order to visualize more intuitively in the rectangular coordinate system, two features are arbitrarily selected from the four features and plotted in the two-dimensional coordinate system.

In [13]:

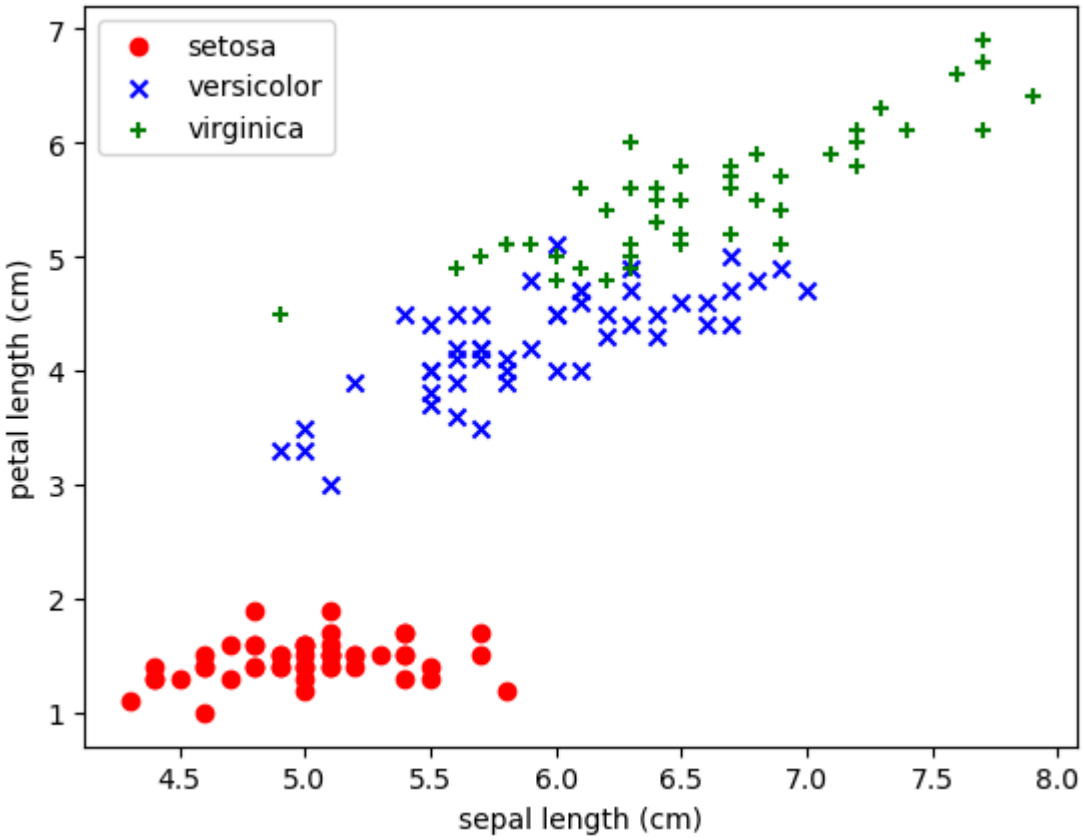
```
import random
import matplotlib.pyplot as plt

feature_1, feature_2 = random.sample([0,1,2,3], 2)
#select 2 random features

plt.scatter(X[0:50,feature_1], X[0:50,feature_2], color="red", marker="o", label
#plot the first 50 samples

#complete code here
```

```
#read the tutorials about matplotlib
#plot the points for the other two flowers
plt.scatter(X[50:100,feature_1], X[50:100,feature_2], color="blue", marker="x",
plt.scatter (X[100:150,feature_1], X[100:150,feature_2], color="green", marker="
plt.xlabel(iris.feature_names[feature_1])
plt.ylabel(iris.feature_names[feature_2])
plt.legend()
plt.show()
```



In [14]:
```
#complete code here
#using a logistic regression model to train the iris data
model = linear_model.LogisticRegression(max_iter=1000)
model.fit(X, y)
```

Out[14]:
```
LogisticRegression
LogisticRegression(max_iter=1000)
```

In [15]:
```
#complete code here
#print the model coefficients
print("Model coefficients:", model.coef_)

#print the model intercept
print("Model intercept:", model.intercept_)

#calculate and print model accuracy
y_pred = model.predict(X)
print("Model accuracy:", accuracy_score(y, y_pred))
```

```
Model coefficients: [[-0.42468882  0.96526487 -2.51571594 -1.08189034]
 [ 0.53472172 -0.31914147 -0.20624079 -0.94393192]
 [-0.1100329  -0.6461234   2.72195674  2.02582227]]
Model intercept: [  9.85904927   2.22777362 -12.08682289]
Model accuracy: 0.9733333333333334
```

In order to visualize the classification results intuitively, 2 and 3 features are selected from [0,1,2,3] four features for training to facilitate drawing

In [17]:
```
#only use two features to train a logistic regression model
feature_1 = 2
feature_2 = 3

#complete code here
#load the original data into X
#put the feature_1 and feature_2 data into X_2
#load the labels into y
X = iris.data
X_2 = X[:, [feature_1, feature_2]]
y = iris.target
```

In [18]:
```
#complete code here
#train a new model using X_2 and y
from sklearn.linear_model import LogisticRegression
model_2 = LogisticRegression()
model_2.fit(X_2, y)
```

Out[18]:
```
▾ LogisticRegression
LogisticRegression()
```

In [19]:
```
#use meshgrid() function to generate two network matrices
h = .02
x_min, x_max = X[:, feature_1].min() - .5, X[:, feature_1].max() + .5
y_min, y_max = X[:, feature_2].min() - .5, X[:, feature_2].max() + .5
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
```

In [20]:
```
cls = np.c_[xx.ravel(), yy.ravel()]
```

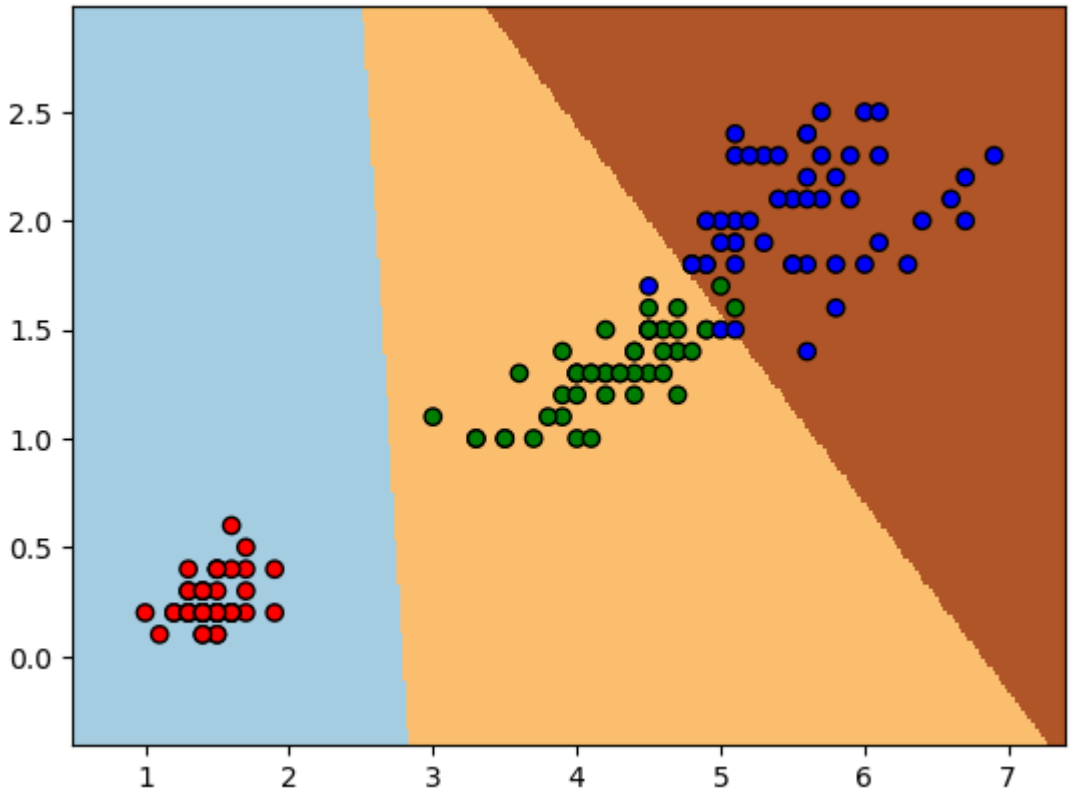
In [21]:
```
pre_cls = model_2.predict(cls)
```

In [22]:
```
pre_cls = pre_cls.reshape(xx.shape)
```

In [23]:
```
plt.pcolormesh(xx, yy, pre_cls, cmap=plt.cm.Paired)

#complete code here
#plot the points of feature_1 and feature_2
from matplotlib import colors
plt.scatter(X[:, feature_1], X[:, feature_2], c=y,
            cmap=colors.ListedColormap(['red', 'green', 'blue']), edgecolors='k')
```

Out[23]:
```
<matplotlib.collections.PathCollection at 0x1883ea42de0>
```



NOTE: 1. When you finish, please use 2pdf.com to transform this ipynb to pdf, and submit it on LMO. 2. Ingore the 'unmatched pages' warning during submiting.